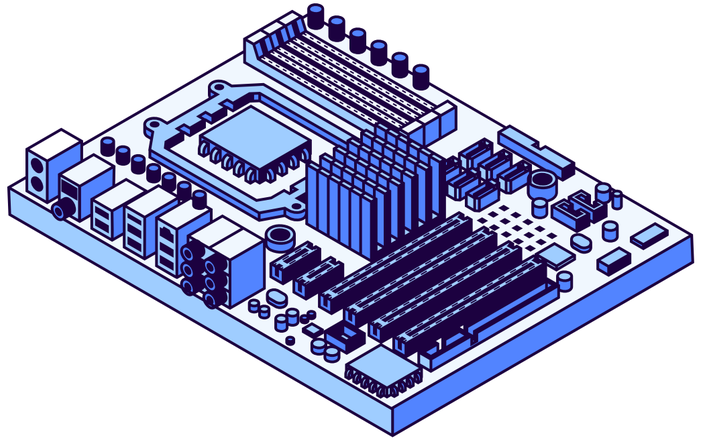
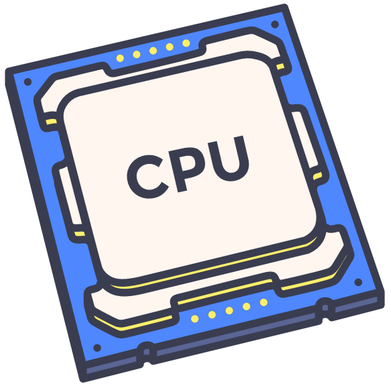
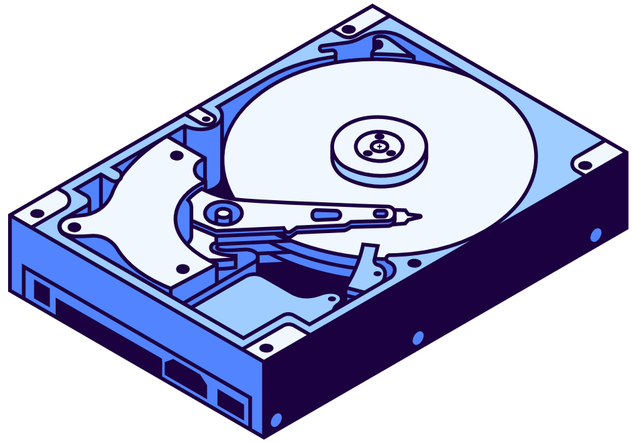
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
GBC045 - Sistemas Operacionais
Prof. Dr. Marcelo Zanchetta do Nascimento



CAPÍTULO 5

ESCALONAMENTO DE PROCESSOS





INTEGRANTES:
Diego De Miranda Da Silva - 12311BCC060
Igor Oliveira Souza e Silva - 12311BCC017
João Pedro Costa Baroni - 12311BCC035
Leonardo Henrique Carvalho Oliveira - 12311BCC003
Moisés Marques de Oliveira - 12021BCC031

SUMÁRIO

Perguntas 3
5.0 Por que gerenciar processos e tempo? 4
5.1 Escalonamento: Quem roda e quando? 5
5.2 Interrupções de relógio e gerenciamento de tempo 6
5.3 Objetivos do escalonador 11
5.4 Escalonamento Tradicional do Unix (SVR3/4.3BSD) 12
5.5 O escalonador SVR4 17
5.6 Melhorias de escalonamento do Solaris 2.X 23
5.7 Escalonamento em Mach 32
5.8 Escalonadores em Sistemas UNIX 33
5.9 Outras abordagens 40
5.10 Conclusão 41
Referências Bibliograficas 42

PERGUNTAS

1. Como os diversos mecanismos do sistema se integram e atuam por meio do escalonador para gerenciar tempo de uso CPU de forma otimizada?

2. Quais são as principais limitações do escalonador tradicional do UNIX (SVR3/4.3BSD) que o tornam inadequado para aplicações em tempo real, e como essas limitações impactam o desempenho do sistema?

3. Como o SVR4 permite o suporte a múltiplas classes de processos, como tempo real e time-sharing?

4. Como lidar com o problema de inversão de prioridade?

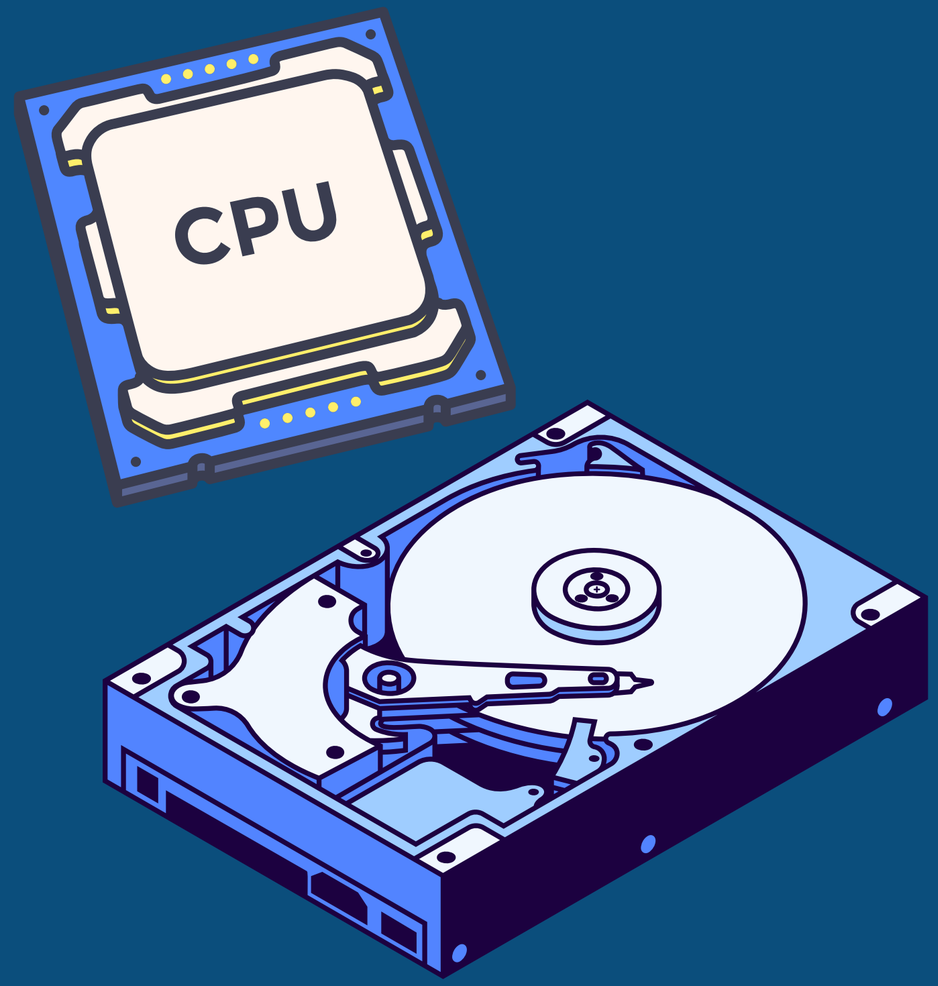
5. Como o Digital UNIX garante a previsibilidade e o desempenho de aplicações em tempo real nos ambientes multiprocessados, mesmo compartilhando a CPU com processos interativos?

5. POR QUE GERENCIAR PROCESSOS E TEMPO?

INTRODUÇÃO AO ESCALONAMENTO (SCHEDULING)
A **CPU** é um recurso compartilhado e o sistema operacional precisa decidir qual processo será executado e por quanto tempo.

Escalonamento é o processo pelo qual o kernel do UNIX decide qual processo (programa em execução) deve usar a CPU (Unidade Central de Processamento) em um determinado momento.

Em sistemas multitarefa como o UNIX, múltiplos processos competem pelos recursos da CPU. **O escalonador** é o componente do kernel responsável por gerenciar essa competição de forma justa e eficiente.

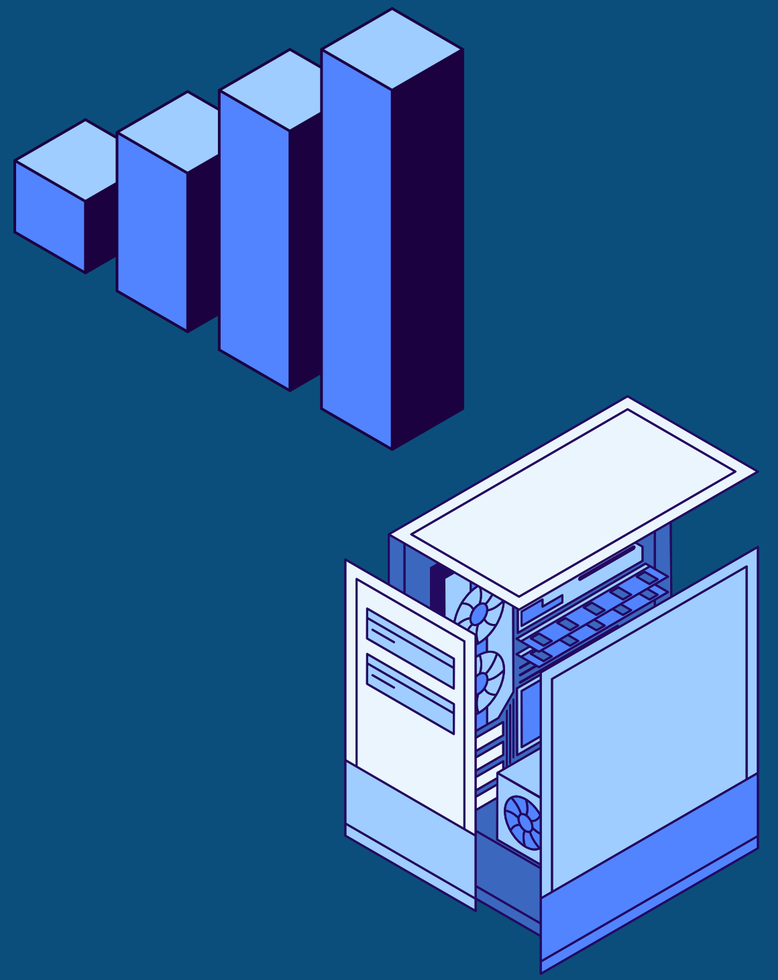


5.1 ESCALONAMENTO: QUEM RODA E QUANDO?

O ESCALONADOR (SCHEDULER):
Ele mantém uma ou mais filas de processos prontos para executar. Quando a CPU fica disponível (seja porque o processo atual terminou, bloqueou ou teve seu tempo esgotado), o escalonador seleciona o próximo processo da fila de prontos para executar.

TROCA DE CONTEXTO (CONTEXT SWITCH):
Quando o escalonador decide trocar o processo em execução, o kernel precisa salvar o estado completo do processo atual (registradores da CPU, contador de programa, ponteiro de pilha, etc.) e carregar o estado do próximo processo selecionado.

Essa operação tem um custo (*overhead* que é uma operação cara em termos de desempenho*)* e o escalonador tenta minimizá-la, mas é essencial para a multitarefa preemptiva.

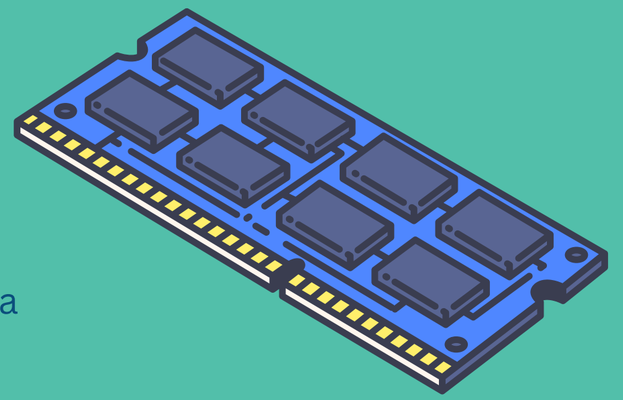


5.2 INTERRUPÇÕES DE RELÓGIO E GERENCIAMENTO DE TEMPO

Interrupções são sinais enviados à CPU por hardware ou software, indicando que um evento que requer atenção imediata ocorreu. Elas "interrompem" a execução normal do programa atual para que o kernel possa lidar com o evento.

TIPOS DE INTERRUPÇÕES E SEU FUNCIONAMENTO:
Interrupções de Hardware: Geradas por dispositivos externos à CPU (controladoras de disco, placas de rede, teclado, temporizador - timer). São assíncronas, ou seja, podem ocorrer a qualquer momento, independentemente do que a CPU está executando.

Interrupções de Software (Traps ou Exceções):
Geradas pela própria execução de uma instrução pela CPU.
São síncronas, ocorrem como resultado direto da execução do código.



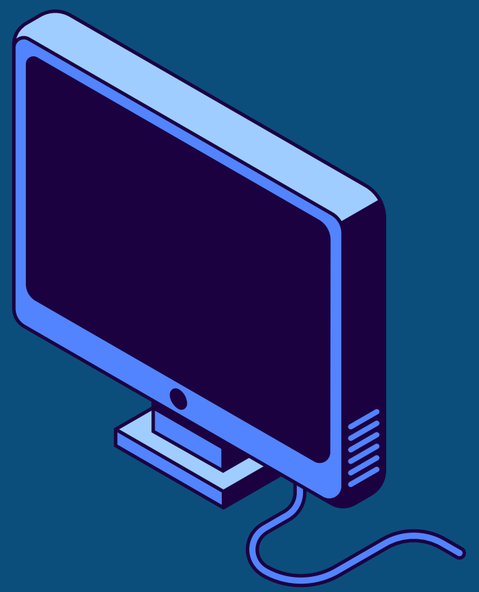
Quando uma interrupção ocorre, a CPU termina a instrução atual (se possível), salva informações essenciais sobre o estado atual (como o contador de programa e flags) na pilha do kernel. A CPU identifica o tipo de interrupção (usando o sinal recebido ou informações do barramento).

5.2 INTERRUPÇÕES DE RELÓGIO E GERENCIAMENTO DE TEMPO

GERENCIAMENTO DE TEMPO (TIME MANAGEMENT):
O gerenciamento de tempo no UNIX refere-se a como o kernel mantém o controle do tempo e fornece serviços relacionados ao tempo para os processos do sistema.

Os objetivos são manter a noção da hora atual (data e hora do dia), medir intervalos de tempo, fornecer a base para o escalonamento preemptivo (através de interrupções de temporizador), permitir que processos durmam (pausem) por um período específico, agendar tarefas para execução futura, contabilizar o uso de CPU por processo, entre outros.

Algumas dessas tarefas não precisam ser executadas a cada tique-taque. A maioria dos sistemas UNIX define uma noção de um tique-taque principal, que ocorre uma vez a cada n ticks do relógio, onde n depende do variante UNIX específico.

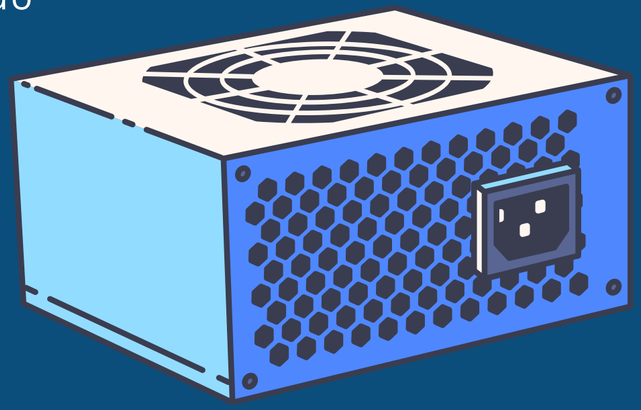


5.2.1 CALLOUTS

Callouts (ou timers de kernel) são um mecanismo fundamental em sistemas UNIX e derivados, como o Linux, que permitem ao kernel agendar a execução de uma função específica para um momento futuro. *Callouts* são consideradas operações normais do kernel e não devem executar em prioridade de interrupção. Portanto, o tratador da interrupção de relógio não invoca diretamente os *callouts*.

Isso é crucial para implementar *timeouts*, tarefas periódicas, *polling* e outras operações dependentes do tempo dentro do próprio kernel. Gerenciar um grande número de *callouts* pendentes de forma eficiente é um desafio de desempenho, pois a verificação precisa ser rápida (ocorre a cada tick).

Existem várias maneiras de implementar a lista de callouts e muitas implementações fornecem uma otimização quando nenhuma outra interrupção está pendente quando o tratador primário termina. Neste caso, o tratador do relógio diretamente baixa a prioridade de interrupção e invoca o tratador de *callout*.



5.2.2 ALARMES DE PROCESSO

Alarmes de processo são um mecanismo fornecido pelo kernel do UNIX (e sistemas derivados como Linux) que permite a um processo solicitar o envio de um sinal para si mesmo após um determinado intervalo de tempo ter decorrido.

TIPOS DE ALARMES:
O UNIX distingue três formas de medir o tempo para esses alarmes, cada uma associada a um sinal específico:

Tempo Real (Real Time): Baseado no tempo de relógio ("*wall-clock time*"), o tempo real que se passou no mundo exterior, independentemente do estado do processo (se estava executando, dormindo, esperando por E/S, etc.).

Tempo Virtual (Virtual Time):
Baseado apenas no tempo em que o processo esteve efetivamente utilizando a CPU em modo usuário.

Tempo de Profiling (Profiling Time):
Baseado no tempo total em que o processo esteve utilizando a CPU, somando o tempo em modo usuário e o tempo em modo sistema (kernel) gasto em nome do processo.

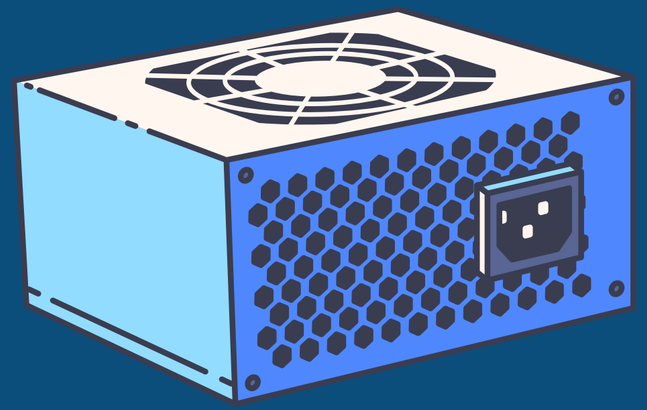


5.2.2 ALARMES DE PROCESSO

LIMITAÇÕES:
É crucial entender que a entrega dos sinais de alarme não é perfeitamente precisa. Podem ocorrer **atrasos em alarmes de tempo real (SIGALRM),** pois embora o kernel detecte a expiração do tempo real com relativa precisão (limitada pela frequência do *timer interrupt* do sistema), a *entrega* do sinal SIGALRM ao processo pode ser atrasada.

Da mesma forma pode ocorrer com a **precisão dos alarmes virtuais e de profiling (SIGVTALRM, SIGPROF):** Embora menos afetados por atrasos devido à carga do sistema (pois medem o tempo de CPU do próprio processo), sua precisão é inerentemente limitada pela granularidade do "tick" do sistema operacional.

Em resumo, os alarmes de processo são uma ferramenta poderosa para controle temporal dentro de aplicações UNIX, mas os desenvolvedores devem estar cientes das diferentes bases de tempo (real, virtual, profiling) e das limitações inerentes à precisão e pontualidade da entrega dos sinais.



5.3 OBJETIVOS DO ESCALONADOR



Figura 01. Objetivos do Escalonamento da CPU

5.4 ESCALONAMENTO TRADICIONAL DO UNIX (SVR3/4.3BSD)

Usado em ambientes interativos de compartilhamento de tempo. Baseado em prioridades dinâmicas:

- Processo de maior prioridade é executado (preempção se necessário).
- Divisão de tempo preemptiva para processos de mesma prioridade.

Kernel não preemptivo: processos em modo kernel não podem ser interrompidos. Favorece processos I/O-bound e evita inanição com recálculo dinâmico de prioridades.

5.4.1 PRIORIDADES DE PROCESSOS

COMO FUNCIONAM AS PRIORIDADES NO UNIX TRADICIONAL

Faixa:
0-127 (menor valor = maior prioridade):
0-49 (kernel); 50-127 (modo usuário).

Estrutura proc: p_pri: prioridade atual (ajustada temporariamente no kernel, ex.: 28 para entrada de terminal). **p_usrpri:** prioridade em modo usuário. **p_cpu:** uso recente de CPU; **p_nice:** fator ajustável (0-39, padrão 20). Fórmula (4.3BSD): **p_usrpri** = 50 + (p_cpu / 4) + (2 * p_nice).

Favorece I/O-bound (alta prioridade); CPU-bound têm baixa prioridade. Decaimento de p_cpu: SVR3 (fator fixo 1/2); 4.3BSD: decay = (2 * load_average) / (2 * load_average + 1).

5.4.2 IMPLEMENTAÇÃO DO ESCALONADOR

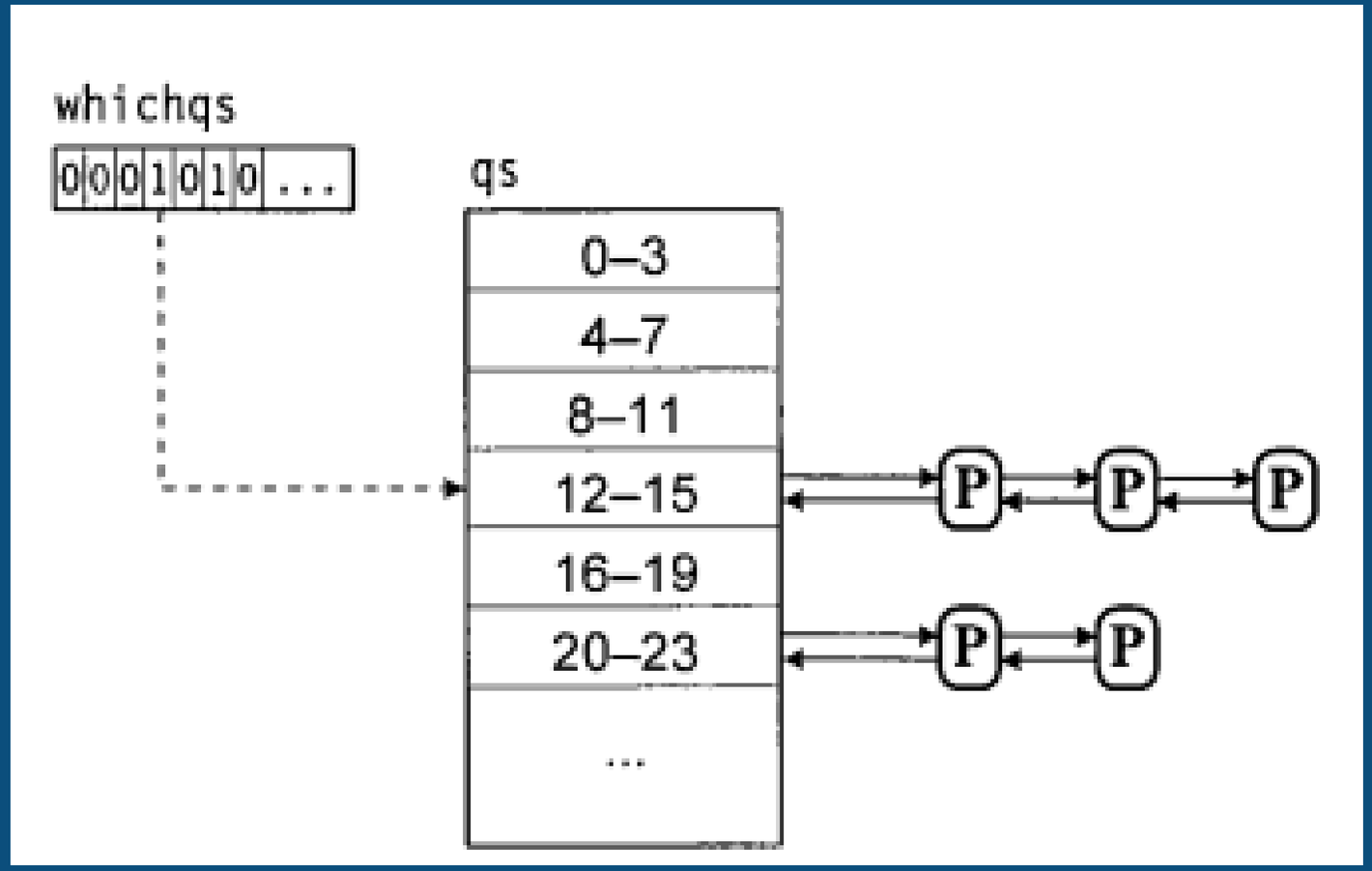


Figura 02. Estruturas de Dados do Escalonador BSD

5.4.3 MANIPULAÇÃO DA FILA DE EXECUÇÃO

Processo de maior prioridade executa, exceto em modo kernel. Quantum: 100 ms (4.3BSD) para processos na mesma fila. A cada 100 ms, roundrobin() (via callout) escalona o próximo processo da fila.

Recálculo de prioridades:
schedcpu(): a cada segundo, remove e reinsere processos em filas conforme nova prioridade.
Relógio: recalcula prioridade do processo atual a cada 4 ticks.

Trocas de contexto:
Voluntárias (bloqueio ou término, via swtch()).
Involuntárias (prioridade maior detectada): flag runrun acionada, swtch() ao retornar ao modo usuário.

5.4.4 ANÁLISE

Eficaz para compartilhamento de tempo, favorecendo I/O-bound e evitando inanição.

Limitações (inadequado para tempo real):

- Não garante tempos de resposta (ex.: 15 fps constantes em vídeos).
- Inversão de prioridade: kernel não preemptivo causa atrasos imprevisíveis. Não escala bem: recálculo de prioridades a cada segundo é ineficiente com muitos processos.
- Não garante alocação fixa de recursos a processos ou grupos.
- Pouco controle de prioridade: nice é simplista. Necessidade de redesign para suportar aplicações em tempo real em sistemas modernos (SVR4, Solaris 2.x, OSF/1).

5.5 O ESCALONADOR SVR4

- Desenvolvido para superar limitações dos sistemas UNIX anteriores.
- Suporta múltiplas classes de escalonamento:
- Time-sharing (tempo compartilhado)
- Real-time (tempo real)
- Utiliza arquitetura modular com rotinas específicas por classe.
- Prioridades de 0 a 159 (valores maiores = maior prioridade).

5.5.1 A CAMADA INDEPENDENTE DE CLASSE & 5.5.2 INTERFACE PARA AS CLASSES DE ESCALONAMENTO

- 160 níveis de prioridade, cada um com sua fila; valores mais altos = maior prioridade.
- Kernel preemptivo com pontos seguros para troca de contexto usando o flag kprunrun.
- Interface genérica de agendamento: cada classe implementa suas funções via classfuncs.
- Processos herdam classe do pai, podendo mudar com priocntl.

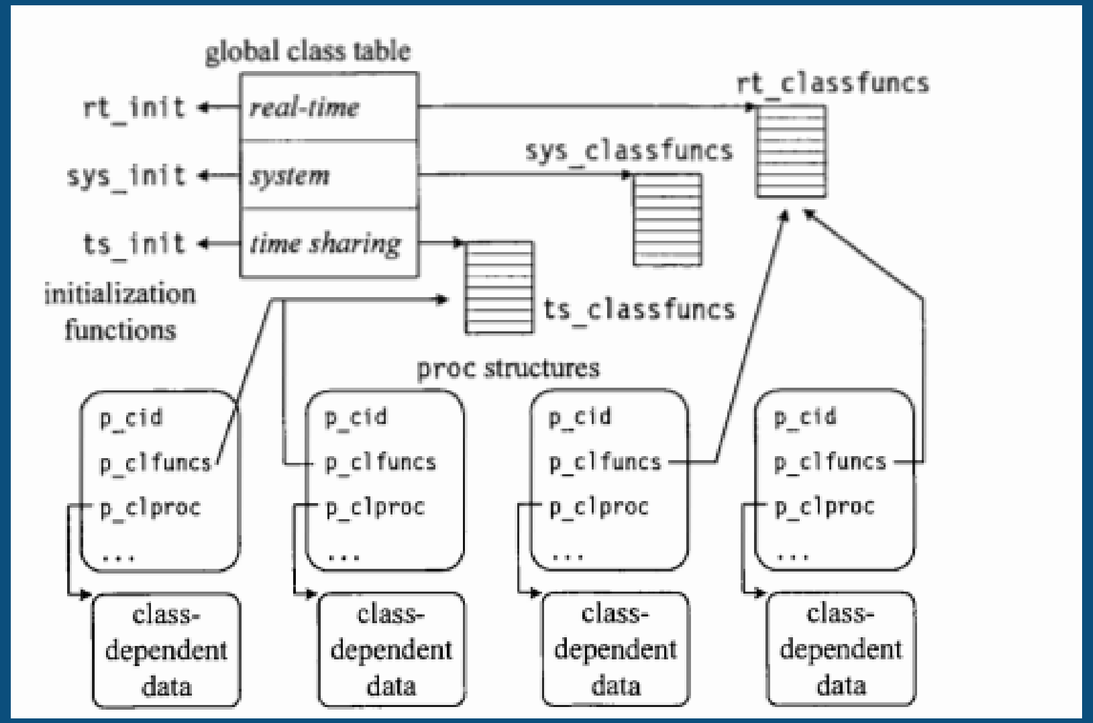


Figura 03. Interface dependente

5.5.3 A CLASSE TIME – SHARING

- Intervalo de prioridade: 0 a 59.
- Escalonamento dinâmico e orientado a eventos.
- Política de round-robin entre processos de mesma prioridade.
- Ajuste de prioridade baseado no comportamento:
- Se consome todo o quantum → prioridade reduzida.
- Se se bloqueia ou é leve no uso de CPU → prioridade aumenta.
- Fatia de tempo maior para prioridades mais baixas

5.5.4 A CLASSE REAL-TIME & 5.5.5 A CHAMADA PRIOCNTL

- Intervalo de prioridade: 100 a 159 (superior à do kernel).
- Prioridade e quantum fixos.
- Controle rígido via chamada priocntl:
  Alterar classe de escalonamento.
  Ajustar ts_upri (time-sharing).
  Definir prioridades e quanta (real-time).
  Consultar parâmetros atuais.

5.5.6 ANALISE E LIMITAÇÕES DO SVR4

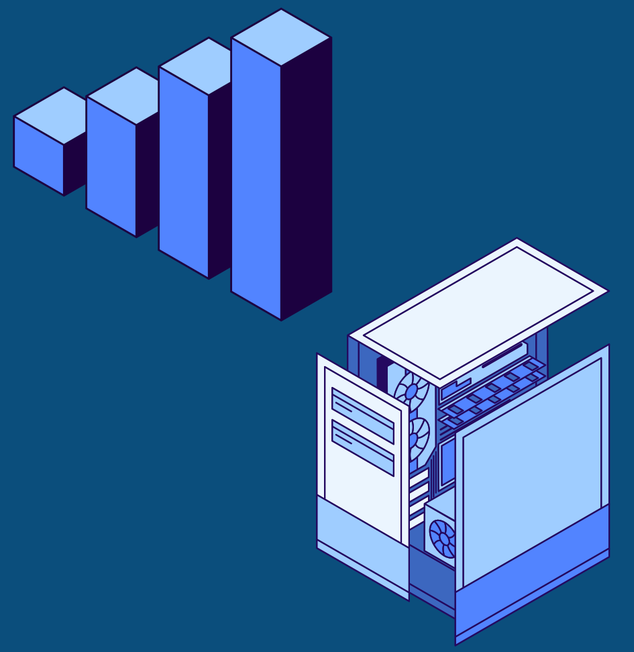
- Mudança de classe requer privilégios de superusuário.
- Sem suporte a deadline scheduling.
- Kernel não totalmente preemptivo, gerando possíveis atrasos.
- Boa modularidade, mas exige ajustes manuais e não é ideal para sistemas críticos com tempo real estrito.

CONCLUSÃO

- Como o SVR4 permite o suporte a múltiplas classes de processos, como real time e time-sharing?

5.6 MELHORIAS DE ESCALONAMENTO DO SOLARIS 2.X

- Sistema operacional multithread e simétrico-multiprocessador.
- Melhora a arquitetura básica de escalonamento do SVR4 em diversos aspectos.
- Escalonador ideal para processamento em tempo real.
    - Reduz latência de despacho

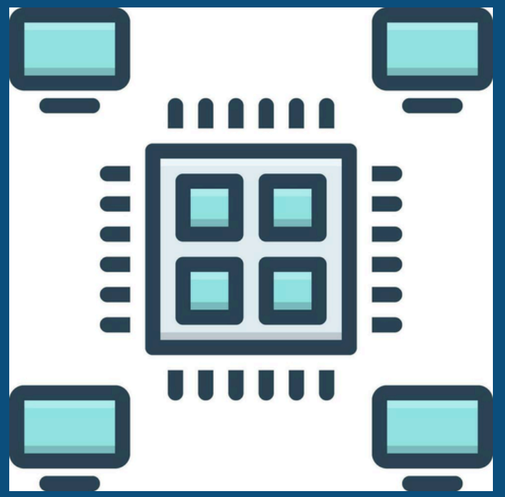


5.6.1 KERNEL PREEMPTIVO

- Mudança radical em relação ao kernel do UNIX
- Consequências:
  - Estruturas de dados do kernel devem ser protegidas por objetos de sincronização
  - Interrupções tratadas por threads especiais que podem bloquear recursos
  - Poucos segmentos de código não preemptivo

5.6.2 SUPORTE PARA MULTIPROCESSAMENTO

- Única fila de despacho
- Threads podem ser fixadas a processadores
- Interrupções entre processadores
- Variáveis de escalonamento
  - cpu_thread: thread sendo executada
  - cpu_dispthread: última thread executada
  - cpu_ idle: thread ociosa
  - cpu_runrun: Sinalizador de preempção usado para threads de tempo compartilhável
  - cpu_kprunrun: Sinalizador de preempção usado por threads de tempo real.
  - cpu_ chosen_ level: Armazena a prioridade da thread que irá preemptar a thread atual



5.6.3 ESCALONAMENTO ESCONDIDO

- O kernel executa tarefas em nome de threads, ignorando prioridade
- Problema do SRV4 com STREAMS
- Solução do Solaris
  - Processamento de STREAMS movido para threads do kernel



5.6.4 INVERSÃO DE PRIORIDADE

- Ocorre quando processo de baixa prioridade bloqueia uma de alta
- Solução - Herança de prioridade
  - Transferência de prioridade
  - Transitividade

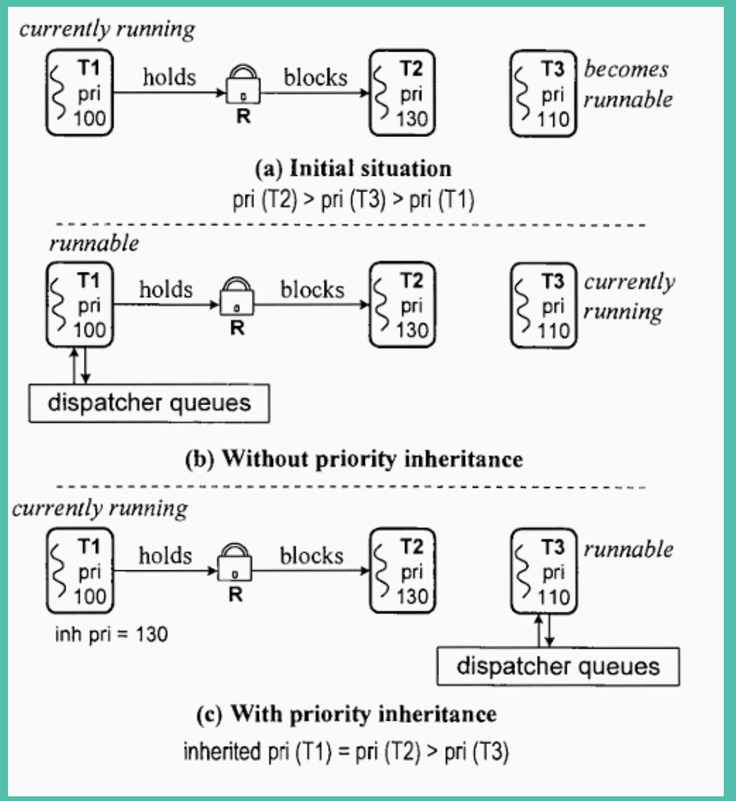


Figura 04. Inversão de prioridade simples

5.6.5 IMPLEMENTAÇÃO DA HERANÇA DE PRIORIDADE

- Threads possuem duas prioridade
  - Global(gp) - classe de escalonamento
  - Herança(ip) - depende das interações
- Função pi_willto()
- Propagação até encontrar thread não bloqueada

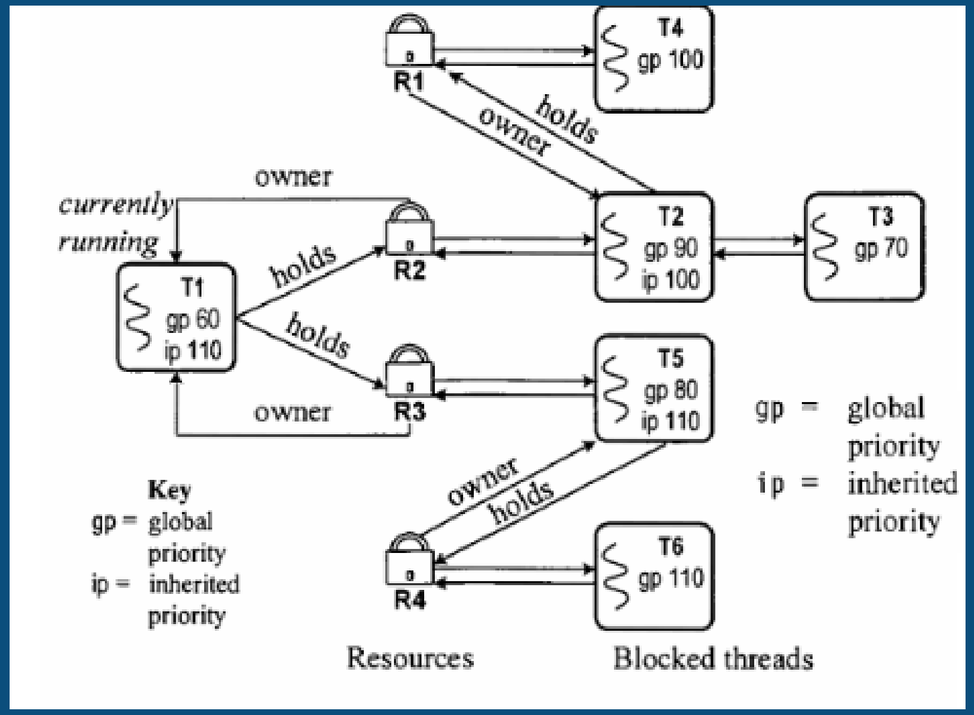


Figura 05. Herança de prioridade

5.6.6 LIMITAÇÕES DA HERANÇA DE PRIORIDADE

- Não se aplica para todos os tipos de sincronização
  - Dono do recurso precisa ser conhecido
  - Funciona bem com mutexes
  - Com locks não é ideal



5.6.7 TURNSTILES

- Usadas gerenciar objetos de sincronização no kernel
- É uma abstração
- Alocados dinamicamente sob demanda

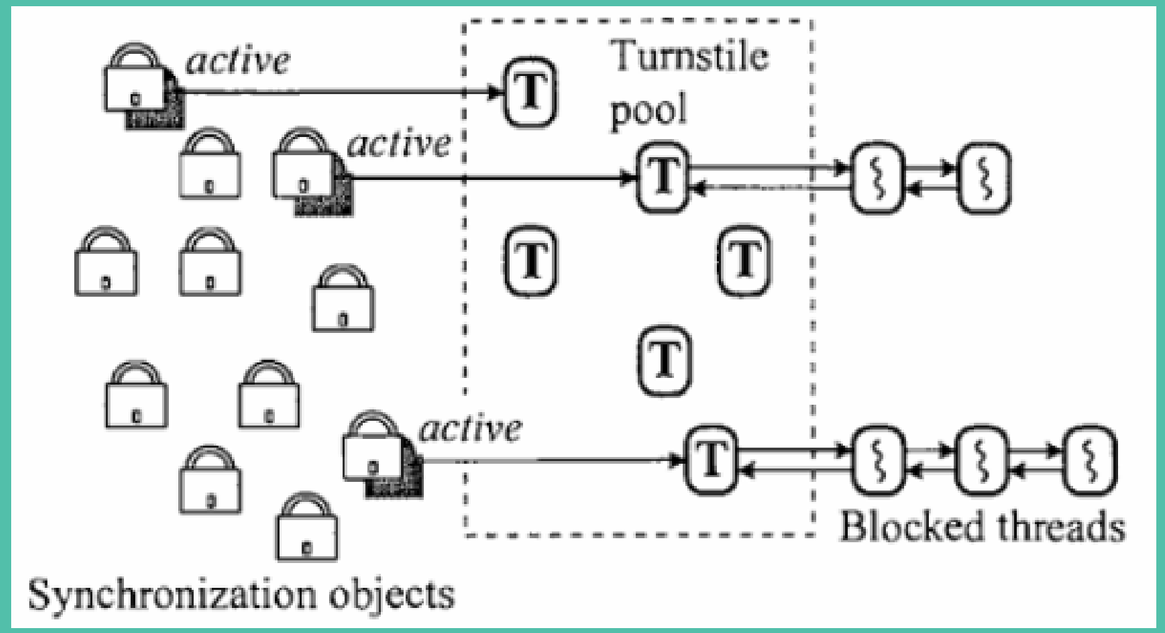


Figura 06. Turnstilels

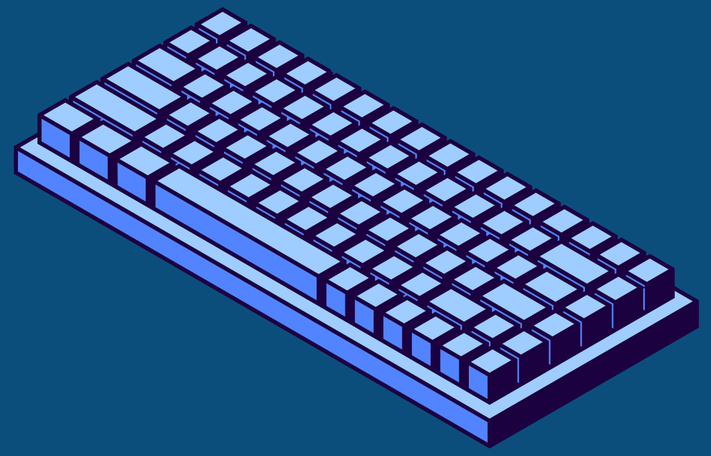
CONCLUSÃO – LIDANDO COM O PROBLEMA DE INVERSÃO DE PRIORIDADE

- Herança de prioridade transitiva
- Turnstiles
- Reduzindo o tempo de threads de alta prioridade ficam bloqueadas



5.7 ESCALONAMENTO EM MACH

- Threads como unidade básica de escalonamento
- Prioridade de execução
- Quantum fixo
- Handoff scheduling

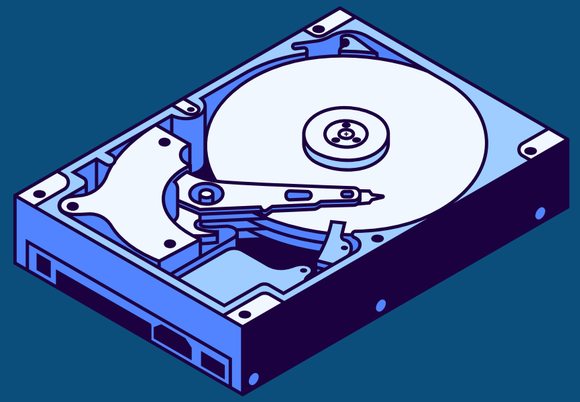


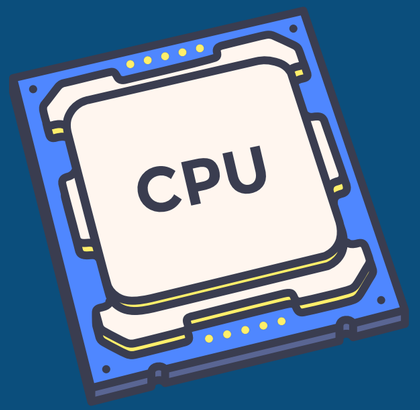
5.8 ESCALONADORES EM SISTEMAS UNIX

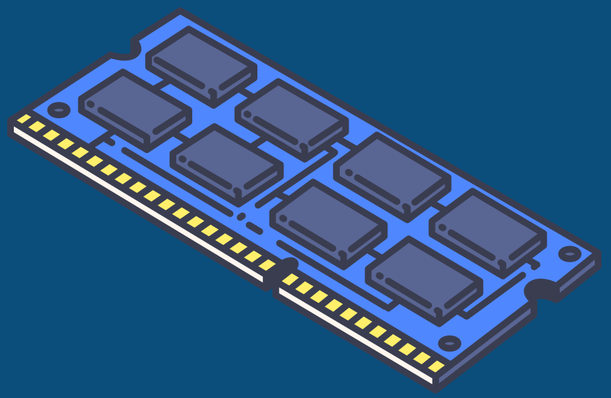
- O que é escalonamento?
- Como o Digital UNIX garante a previsibilidade e o desempenho de aplicações em tempo real nos ambientes multiprocessados, mesmo compartilhando a CPU com processos interativos?

5.8.1 O QUE É ESCALONAMENTO?

- Escalonamento é decidir qual processo será executado em certo momento.
- Crucial para o desempenho e previsibilidade do funcionamento dos sistemas.
- Existem diferentes políticas de escalonamento adotadas pelos escalonadores.







5.8.2 ESCALONADOR NO DIGITAL UNIX

- **SCHED_OTHER:** Compartilhamento de tempo
- **SCHED_FIFO:** Primeiro a chegar é o primeiro a sair
- **SCHED_RR:** round-robin

Possuem prioridade compartilhada que vai de 0 a 63, quanto maior o número maior a prioridade, prioridades acima de 19 precisam de privilégios de super usuário.
O escalonador executa primeiro os processos com maior prioridade.

5.8.2.1 TIME-SHARING

- Usa o quantum, uma unidade de tempo definida pelo SO, para dividir o tempo da CPU.
- Tem priorização com base no **nice value** (-20 a +20, os numeros negativos definem prioridades de 20 a 29)
- Quanto mais CPU é usada pelo processo menor é sua prioridade.

5.8.2.2 FIFO (FIRST-IN FIRST-OUT)

- Não possui o quantum, executa os processo até o fim ou até que uma preempção o pare.
- Prioridade fixa de 32 a 63.
- Apenas super usuário pode definir prioridade acima de 19.

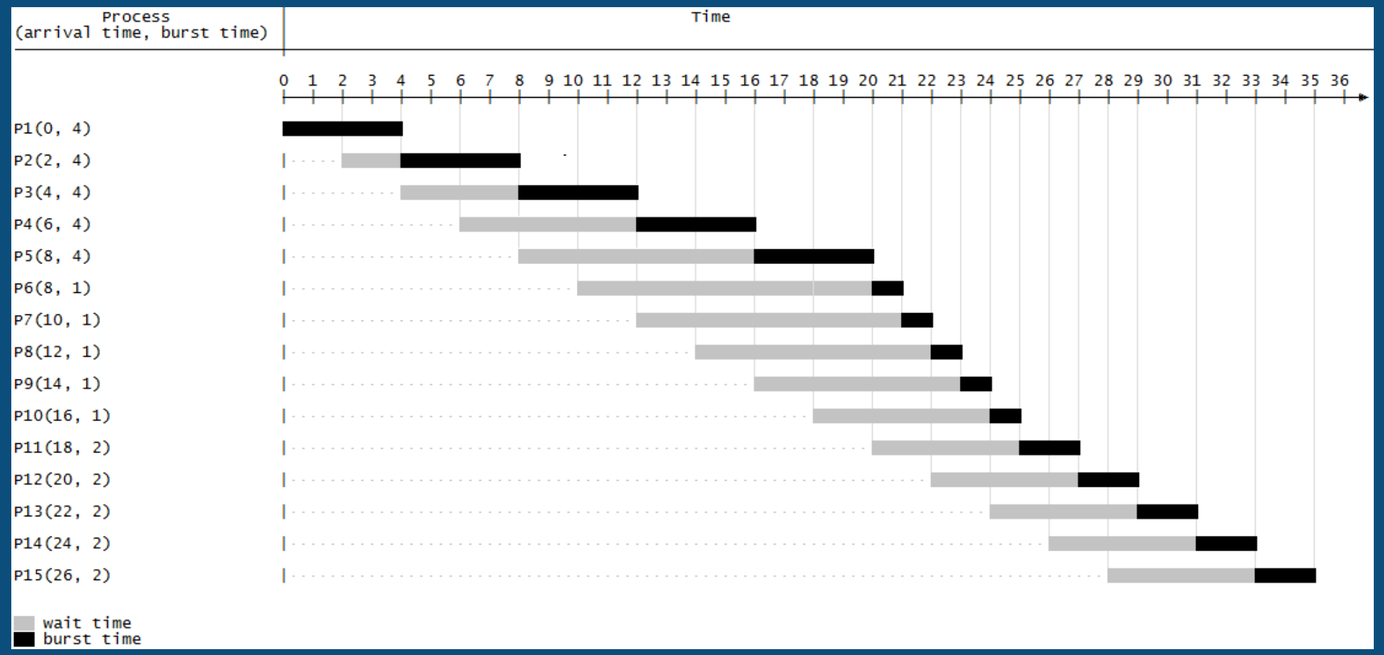


Gráfico 1 – Escalonamento de processos com manuseio de interrupções de relógio

5.8.2.3 ROUND-ROBIN

- Utiliza de um quantum como o time-sharing.
- Prioridade fixa de 32 a 63.
- Apenas super usuário pode definir prioridade acima de 19.

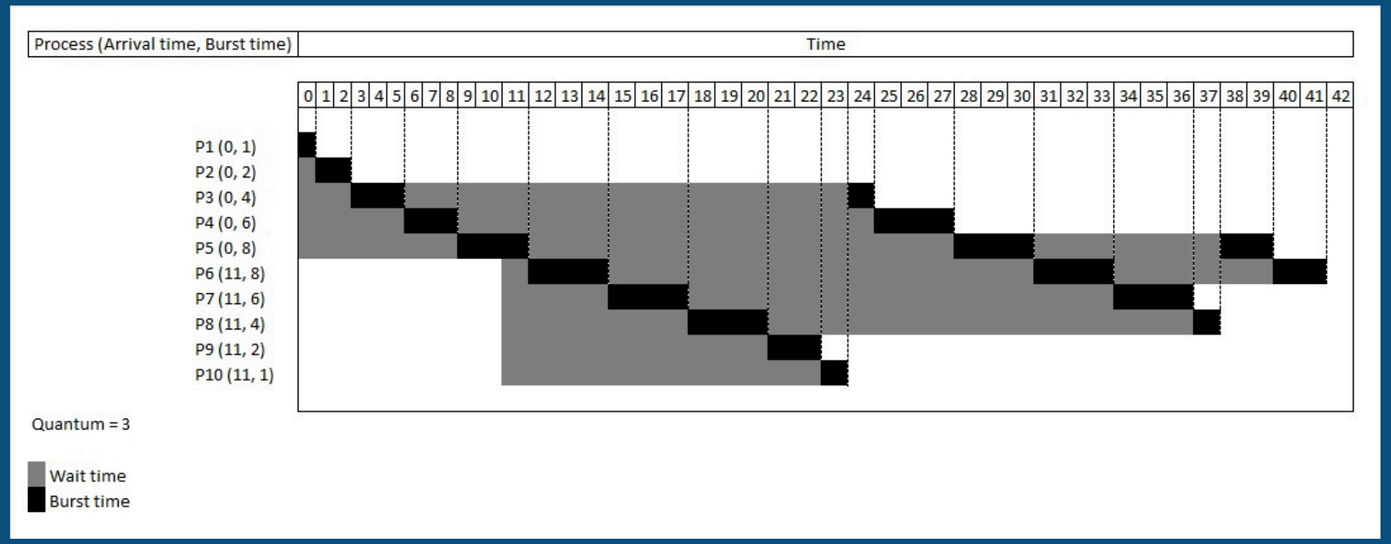
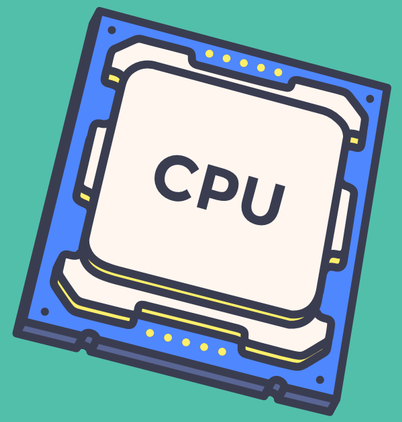


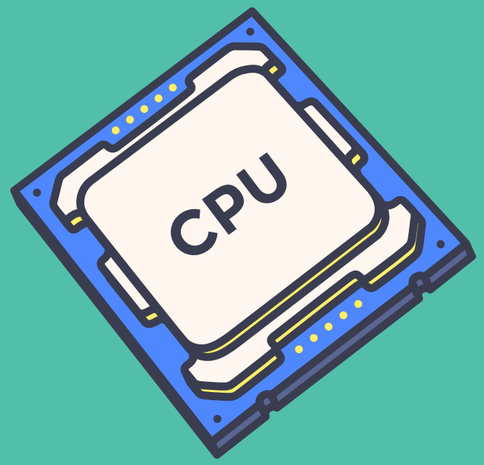
Gráfico 2 – Escalonamento de processos com quantum de tempo nominal igual a 3

5.8.3 MULTIPROCESSADORES



- Uso de **filas locais** por processador para evitar gargalos.
- **Afinidade suave:** processo tende a voltar ao mesmo CPU.
- **Balanceamento dinâmico** entre CPUs.

O Digital UNIX também pode trabalhar com o **bind_to_cpu()** para que a thread sempre use o mesmo processador.



5.9 OUTRAS ABORDAGENS

Fair-Share: divide CPU por grupo (ex: usuários/sessões).
Deadline-Driven (ex: FORTUNIX): prioridade aumenta conforme o prazo se aproxima.
Três Níveis: tarefas divididas em:

- Isocrônicas
- Tempo real
- Gerais

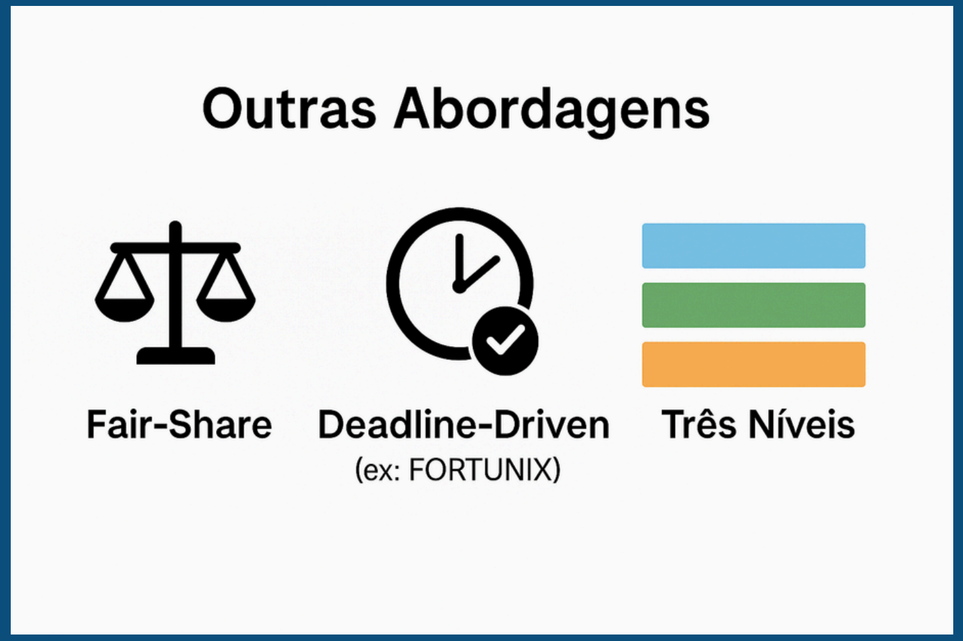


Figura 07 -Abordagens de escalonamento: Fair-Share, Deadline-Driven e Três Níveis

5.10 CONCLUSÃO

Como o Digital UNIX garante a previsibilidade e o desempenho de aplicações em tempo real nos ambientes multiprocessados, mesmo compartilhando a CPU com processos interativos?

Digital UNIX garante desempenho e previsibilidade com:

- Prioridades fixas controláveis via POSIX
- Separação entre classes de carga
- Eficiência em ambientes multiprocessados

REFERÊNCIAS BIBLIOGRÁFICAS

BACH, M. The design of the Unix operating system. Englewood Cliffs: Prentice-Hall, 1990.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Fundamentos de sistemas operacionais. 6. ed. Rio de Janeiro: Campus, 2004.

TANENBAUM, A. S.; WOODHULL, A. S. Sistemas operacionais: projeto e implementação. Porto Alegre: Bookman, 2000.

VAHALIA, U. Unix internals: the new frontiers. Englewood Cliffs: Prentice Hall, 1996.

REFERÊNCIA MULTIMÍDIA

ChatGPT - Foram utilizadas prompts para tradução do conteúdo do livro, originalmente em inglês e para sanar dúvidas sobre os conteúdos pesquisados:

- "Explique detalhadamente sobre Escalonamento, Interrupções e Gerenciamento de Tempo no UNIX.
- " Me dê sugestões de uma melhor estrutura de diagramas para ilustrar o luxo de execução de callouts e timing wheel.”
- “Traduza o conteúdo desse pdf sobre sistemas UNIX.”

Youtube: Escalonamento de Processos - SISTEMAS DE COMPUTAÇÃO

Website: Gerenciamento de processos - Celio Normando

Grafico do FIFO: FIFO – Wikipédia, a enciclopédia livre

Grafico do RR: Round-robin – Wikipédia, a enciclopédia livre